

COMP3011 - CWK1

Morgan Thomas
201726385

March 2026

[GitHub Repo](#) [Hosted Documentation](#) [PDF Documentation](#) [Hosted API](#) [Presentation Slides](#)

1 Introduction

1.1 Project Overview

The UK housing market represents one of the largest and most data-rich sectors in the UK economy. Much of this data is analysed using proprietary platforms aimed at large organisations, limiting accessibility for independent developers and researchers. There are, however, many public datasets which contain a lot of the raw information and are often what these expensive services use to. These datasets themselves are not very useful as they are often in a raw format, aren't queryable, or have inconsistencies.

This project presents the design and implementation of a data-driven web API for housing market analytics. The system integrates property transaction data from the Price Paid dataset with Energy Performance Certificates (EPC), storing them in a relational PostgreSQL database. The API enables users to perform analytical queries on housing data, including price statistics, transaction volumes, and regional trends.

The backend system is implemented using FastAPI, providing a RESTful API with strongly typed requests and responses, alongside extensive documentation. To support long-running and computationally intensive tasks, the system uses Celery workers with Redis caching to allow these tasks to be executed asynchronously.

In addition to the analytical function, this API offers secure authentication mechanisms, using JWT-based user authentication and API key generation for programmatic access. The platform also implements usage-based billing where API calls are recorded and aggregated before being sent to Stripe for monthly billing.

The system is designed with modern software engineering practices, including containerised deployment, Kubernetes orchestration, automated CI/CD pipelines, and comprehensive testing using PyTest and TestContainers. These features aim to ensure reliability, scalability, and maintainability.

2 Data Source and Ingestion

2.1 Datasets

This project integrates two publicly available datasets on UK housing to enable property market analysis.

The first dataset is the UK Land Registry Price Paid Data (PPD) [3], which records all property transactions in England and Wales. Each record contains information such as the sale price, transaction date, property type, and address information. The dataset provides historical sales information back to 1995, when reporting became mandatory.

The second dataset used is the Energy Performance Certificate (EPC) dataset [6], which contains information about the energy efficiency of residential properties. EPC records include metrics such as energy efficiency ratings, environmental impact score, and estimated energy cost. By combining this dataset with PPD, the platform is able to provide much greater context for prices and market activity, allowing users to perform more rigorous analysis.

Both datasets are publicly available from the UK government; the URLs are listed below -

- PPD - <https://www.gov.uk/government/statistical-data-sets/price-paid-data-downloads>
- EPC - <https://epc.opendatacommunities.org/domestic/search>

2.2 Data Ingestion

The price paid dataset contains over 28 million transactions, making efficient ingestion an important consideration. To address this, I developed an ingestion tool in Rust.

Rust was selected due to its strong performance characteristics like C or C++, while also offering memory safety guarantees. The ingestion tool fetches the entire dataset file from the government's AWS S3 bucket. It then reads this file and performs high-speed null insertions into the PostgreSQL database.

The Land Registry publishes a new update file every month, which contains all of the new transactions to be added and any amendments or deletions. The Rust script stores a hash in the PostgreSQL database, which is of the most recent file. It periodically pulls the update file to check if it has changed and will ingest it.

For the EPC data, a machine interface is not provided, so the zip file containing all of the certificates must be downloaded. The Rust tool iterates through all of the files in the zip and uploads them in a similar fashion to the PPD data. Since there is no machine interface, either the data must be updated manually by ingesting the entire file again, as they do not provide a periodic difference file for new data.

This approach allows the large datasets to be imported efficiently and kept up to date automatically, where possible.

2.3 Database Schema

The sales are stored in a normalised relational schema consisting of three primary tables (sales, houses, postcodes). Normalisation was used to reduce redundancy and improve query performance. Three more fields have also been derived for the postcodes table: area, outcode, and sector, which allow a finer granularity when searching. A postcode is made up of four parts: the full postcode itself, the area which is the first letter(s), the outcode which is the first letter(s) and number sequence, and the sector which is the outcode plus the first number on the second half. For example, the postcode CH64 1RG has the area CH, outcode CH64 and sector CH64 1. The last two letters do not indicate the postcode's geographical location.

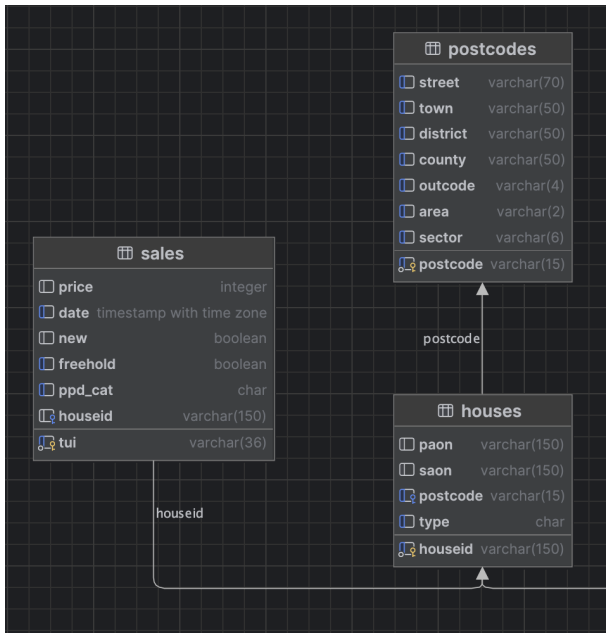


Figure 1: Entity relationship diagram of normalised schema for PPD

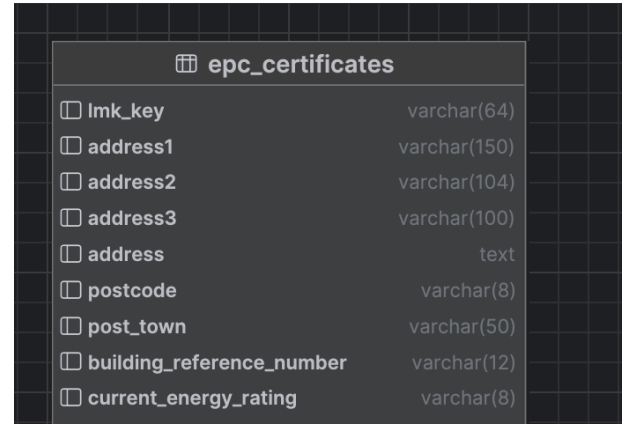


Figure 2: Entity relationship diagram for EPC

The EPC data has no direct link to the PPD data. They both contain address data, but this is quite inconsistent and there is no agreed-upon format. In the UK, there are Unique Property Reference Numbers which assign a unique identifier to every building and installation in the UK, like postboxes and telephone boxes. This allows for universal referencing of the object. EPC data contains the UPRN for each house that has a certificate. The PPD data does not support linking, so it requires the address to be matched by some combination of narrowing down the possible houses by postcode and finding the most similar address. By linking these two datasets, the price paid datasets are able to be associated with a UPRN, which wouldn't normally be possible without paying the Ordnance Survey exorbitant fees. This is done using the below SQL statement, which is run on the first ingest and then on any new transactions added. The PostgreSQL pg_trgm extension is used to compute text similarity between addresses, allowing the closest match to be selected when exact matches are unavailable [7].

```
INSERT INTO uprn_lookup (houseid, uprn)
SELECT
    s.houseid,
    e.uprn
FROM sales s
JOIN houses h ON h.houseid = s.houseid
JOIN LATERAL (
    SELECT uprn
    FROM epc_certificates e
    WHERE e.postcode = h.postcode
    ORDER BY similarity(
        CASE
            WHEN h.paon IS NOT NULL AND h.paon <> ''
```

```

        THEN COALESCE(e.address1, '') || '␣' || COALESCE(e.address2, '')
        ELSE COALESCE(e.address1, '')
    END,
    COALESCE(h.paon, '') || '␣' || COALESCE(h.saon, '')
) DESC
LIMIT 1
) e ON TRUE;

```

3 System Architecture

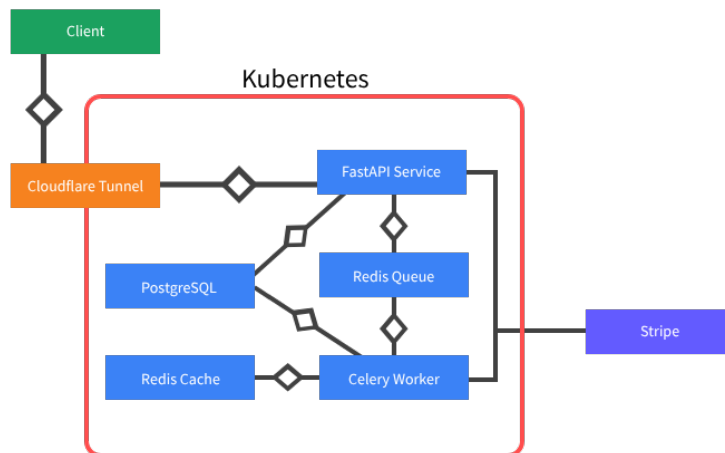


Figure 3: Overall System Architecture

3.1 Overview

The system follows a service-oriented architecture designed to facilitate scalable data analytics and high-performance API responses. The architecture separates the responsibility between multiple components, allowing computationally expensive tasks and background operations to be processed asynchronously without affecting API latency. Without asynchronous processing, heavy aggregations over millions of rows would block the API thread and significantly increase response latency.

3.2 API Layer

The API layer is implemented using FastAPI [8], a modern Python web framework designed for high-performance asynchronous APIs. FastAPI was chosen for its asynchronous request handling, automatic OpenAPI documentation, and strong type validation. These features enable high-performance APIs with minimal boilerplate. There are other Python web libraries like Flask and Django. I found Django came with a lot of boilerplate that would not be used in this application, Flask and Django are also WSGI based libraries meaning requests are synchronous which can lead to worse performance especially in I/O heavy scenarios/

The API server is responsible for authenticating users, validating request parameters, handling all API endpoints, and forwarding tasks to the Celery workers. Keeping the API layer lightweight allows response time to remain low even for the long-running analytical operations.

3.3 Background Tasks

Some of the operations performed by the API are computationally expensive or involve external services. To avoid blocking the API server, these operations are handled using Celery workers [2]. Tasks are called using a library in the API server, which sends a task over a Redis queue. These tasks are then handled by the Celery workers. You can scale the Celery workers independently of the web API, so when a large amount of analytical requests are submitted, or the cache has recently been flushed, like after a new data ingest, it can handle this surge in traffic. Examples of these background tasks include, property data aggregations, API usage logging, and scheduled billing aggregations.

When a request requires heavy computation, the API submits the task to the Redis queue via the Celery library. A worker then executes the task independently and returns the result once complete. This asynchronous architecture improves scalability and ensures that long-running tasks do not degrade the API service. FastAPI offers 'background-tasks' however these run in the webserver and if enough tasks are being run on an instance it will eventually become locked up. By using Celery workers the processing can be scaled independently of the web API.

3.4 Redis Cache

To improve the performance of the data aggregations, the system uses Redis [9] key-value store as a caching layer. When an aggregation job is submitted to the Celery worker, it will first check the cache to see if there is a recent result. If it gets a response, then that is returned instead of running the full aggregation. This greatly improves response time since tasks that use several million rows can take a discernible amount of time for the user. This method caches at the individual aggregation level rather than the request, since different aggregations can be shared between requests, so a request could use partially cached data.

This approach significantly reduces the cost of repeated aggregations and improves overall response time. Using Redis instead of an in-memory cache allows the cache to be shared across multiple Celery workers.

3.5 Authentication and Security

Users are required to authenticate to use the API service. This is so their usage can be monitored, and they can be charged for it. Users sign up with an email, password, and name. They are then registered with Stripe so they can go through Stripe's flow to set up their payment methods. The user's credentials are stored in the PostgreSQL database with a Universal Unique Identifier. Any user data is associated using that ID, like endpoint usage, API tokens, etc. Their password is salted, hashed, and then stored.

When a user logs in, they receive a JSON Web Token [4], which is used to authenticate them against the API service. They also get a refresh token stored in a secure cookie. This is used to get a new JWT when the first one expires. The JWTs are stateless and have short expiry times, so if compromised, attackers have little time to use them. The refresh tokens are stateful and stored in the PostgreSQL database, so they can be revoked; they can only be used to issue a new JWT token, which means they can have much longer expiry times.

Users also have the option to generate API keys. These are used to interact with the aggregation and statistic routes. The API keys do not have access to any of the 'infrastructure' routes like user account or billing. They can only be used to generate aggregations. These keys are generated and stored as hashes. When a request is made with one, it is compared against the DB to validate it. These keys have much longer expiry and can be revoked at any time by the user. These are aimed at instances where the user may want to use an application to interact with the system, like a script to generate further analysis off the data they queried.

3.6 Usage Billing System

Users are charged in tokens for requests they make to certain routes. These need to be logged and registered with Stripe so the user can be charged for them. When a user makes a request to a route that has the billing attribute, a job is sent to the Celery workers to log this usage in the database. This is done with a Celery worker so the I/O operations do not increase the API latency.

Every day at 11 pm, a task is scheduled to take all of the usage from users that day and aggregate it into one charge. This charge is then submitted to Stripe as a metered event. The reason usages are aggregated is to reduce the requests to Stripe since they are rate-limited and making a request after every API call would use a lot of I/O time.

The aggregation of each user's daily usage is stored as an event, so they can be referenced and if a usage hasn't been added to an aggregation, it will be included in the next one. This also allows traceability to what and when a user has been charged for their usage.

At the end of every month, Stripe will automatically charge the user's default payment method with their usage for that month or send them an invoice if there is no payment method on file.

4 Testing Strategy

Ensuring the reliability and correctness of the API was an important part of the development process. It allowed me to develop the application in modules and test them independently to ensure their correctness without needing the full system to be developed. Once the system was developed, end-to-end tests were implemented to validate the behaviour of the entire system.

4.1 Unit Tests

Unit tests were implemented using the PyTest framework. These tests focused on verifying the behaviour of individual components or modules in isolation, ensuring that the core logic functions correctly. All of these tests aimed to validate the nominal behaviour and check behaviour under incorrect parameters and conditions.

The unit test suite covers key areas of the system, including, authentication logic and token generation, user creating and database validation, analytics aggregation functions, and billing aggregation and usage logging functions.

4.2 End-to-End Testing

Whilst the unit tests verify the individual components, it is also important to ensure full system functionality when all the services interact together. To achieve this, end-to-end tests were implemented using TestContainers.

TestContainers allow the creation of temporary containerised services during testing. This allows tests to run as if they were a fully deployed setup. These dependencies included a PostgreSQL database, Redis cache, and Redis queue.

The E2E tests simulate realistic usage, including: user authentication flows, background task execution, database interactions, and result caching behaviour.

5 Deployment and DevOps

The system is designed to support local development and full production deployment using containerisation and modern DevOps. All application components are packaged using Docker, ensuring a consistent environment across machines and scenarios.

For local development, the full system can be launched using a Docker Compose file. This file orchestrates all necessary services, including Redis, PostgreSQL, Celery workers, Celery Beat, and the web API. This approach simplifies development, as the full stack can be started with a single line, and it maintains parity with production [5]. Additionally, a Kubernetes deployment design is available for local use, replicating the cloud version as closely as possible using Kubernetes in Docker (KIND).

In production, the application is deployed using Kubernetes. Kubernetes provides container orchestration, service management, and improved scalability [1]. Each component runs as a pod or set of pods, accessed through a service. These pods can be scaled horizontally and load-balanced through the service, allowing for dynamic scaling of the web API and Celery workers. Furthermore, there's potential for read replicas of PostgreSQL to be deployed to improve statistical queries, as they don't require write operations and free up the main database. External traffic is securely routed through Cloudflare, minimising the attack surface by keeping the Kubernetes cluster isolated from the internet.

A continuous integration and continuous deployment pipeline is implemented using GitHub Actions. This pipeline automatically performs linting, testing, and container building. These actions run on every pull request into the main branch, preventing the request from merging until all are successful. This limits the amount of bad code that enters production. When a release is created on the main branch that invokes an action to build the container and publish it to GitHub Container Repository, it can be pulled by the Kubernetes cluster and deployed to production. The deployed version of this application uses only a subset of the datasets due to limitations of the cloud hosting service. This is because the dataset contains data only from houses in the 'LS' postcode area, which includes Leeds and its suburbs.

6 Limitations and Future Improvements

A key limitation of the current system is the lack of a consistent, unique property identifier across datasets. The UPRN provides a standard way to identify properties, but it is not published on all datasets. As a result, the system relies on postcode and address matching, which can introduce inconsistencies. Future work could integrate paid datasets or services that are able to perform address matching, enabling a more accurate join between proper datasets.

The Ordnance survey publish a dataset where it had the UPRNs and their corresponding latitude and longitude. This could be joined with the governments INSPIRE polygon dataset which has land borders. Doing so would allow the incorporation of property plot size and other geospatial data like flood risk, local school, surrounding green areas, and

Another potential improvement is extending the authentication system into more of an identity provider. While the API currently supports JWT authentication and API keys, expanding this into a centralised identity service would allow greater integration with external systems like MCP clients.

The current analytics layer is implemented in Python, which offers a strong ecosystem and support for data science, but is not always optimal for data aggregation. Future versions of the system could improve performance by introducing more specialised data processing solutions or implementing critical aggregations in high-performance languages.

7 GenAI Usage

Generative AI tools were used throughout the development process as a productivity and problem-solving aid. AI was particularly useful during the early design stage of the project, where it helped to explore possible system architectures and assisted in breaking the web API into logical components and services. It was also used to help me come up with feature ideas that would help me climb up the mark scheme, like usage-based billing and API tokens.

During the actual development, AI was used as a debugging assistant to counter the niche issues and errors that were difficult to resolve through the documentation.

Another area where AI was helpful was creating the test scaffolding. Writing comprehensive tests often involves repetitive steps such as creating mock objects and coming up with the test data. AI was used to assist in drafting these repetitive tests, usually after an initial test had been made.

During deployment, AI assisted with debugging the Kubernetes configuration. Having never used it before and being used to Docker, it allowed me to explore the nuances and understand the feature whilst actively using it on my project instead of having to follow a tutorial performing unrelated steps at points.

Overall, AI acted primarily as a development assistant and reasoning aid, helping accelerate development while all design decisions and implementations were ultimately reviewed and validated manually.

References

- [1] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *Communications of the ACM*, 59(5):50–57, 2016.
- [2] Celery Project. Celery: Distributed task queue documentation. <https://docs.celeryq.dev/>, 2024. Accessed: 2026-02-20.
- [3] HM Land Registry. Price paid data. <https://www.gov.uk/government/statistical-data-sets/price-paid-data-downloads>, 2024. Accessed: 2026-02-13.
- [4] Michael Jones, John Bradley, and Nat Sakimura. Json web token (jwt). RFC 7519, Internet Engineering Task Force, 2015.
- [5] Dirk Merkel. Docker: Lightweight linux containers for consistent development and deployment. *Linux Journal*, (239), 2014.
- [6] Ministry of Housing, Communities and Local Government. Energy performance certificates dataset. <https://epc.opendatacommunities.org/>, 2024. Accessed: 2026-02-13.
- [7] PostgreSQL Global Development Group. pg_trgm: Text similarity extension. <https://www.postgresql.org/docs/current/pgtrgm.html>, 2024. Accessed: 2026-02-25.
- [8] Sebastián Ramirez. Fastapi documentation. <https://fastapi.tiangolo.com/>, 2024. Accessed: 2026-02-16.
- [9] Redis Ltd. Redis documentation. <https://redis.io/docs/>, 2024. Accessed: 2026-03-8.

8 Appendix

8.1 GenAI Conversation Examples

Documenting FastAPI Errors

03/13/2026 21:06:58

How do you add documented return erros to fastapi route

For your coursework, this matters. The brief explicitly says your API must "use...



How do you add documented return errors to fastapi route



For your coursework, this matters. The brief explicitly says your API must **“use correct status/error codes, according to industry convention”** and your documentation must **“document authentication process and error codes where applicable”**

COMP3011_Coursework1_Brief__202...

So don't just *return* errors — **document them properly**.

Here's how to do it cleanly in FastAPI.

1 The Basic Way: `responses=` in Your Route Decorator

FastAPI lets you explicitly declare possible error responses.

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel

router = APIRouter()

class UserResponse(BaseModel):
    id: int
    email: str

@router.get(
    "/users/{user_id}",
    response_model=UserResponse,
    responses={
        404: {
            "description": "User not found",
            "content": {
                "application/json": {
                    "example": {"detail": "User with this ID does not exist"}
                }
            },
        },
    },
)
```



```

401: {
    "description": "Unauthorized",
    "content": {
        "application/json": {
            "example": {"detail": "Invalid authentication credentials"}
        }
    },
},
),
)
async def get_user(user_id: int):
    user = fake_db_lookup(user_id)
    if not user:
        raise HTTPException(status_code=404, detail="User with this ID does not exist")
    return user

```

What this does:

- Adds structured error documentation to Swagger/OpenAPI
- Makes examiners happy
- Makes your API professional

2 Better: Define a Standard Error Model

If you want to push toward 70–80+ territory, define consistent error schemas.

```

class ErrorResponse(BaseModel):
    detail: str

```

Then:

```

@router.get(
    "/users/{user_id}",
    response_model=UserResponse,
    responses={
        404: {"model": ErrorResponse, "description": "User not found"},
    },
)

```

```
    401: {"model": ErrorResponse, "description": "Unauthorized"},
  },
)
```

Now your API:

- Has consistent error structure
- Is properly typed
- Looks like you know what you're doing

3 For Validation Errors (422)

FastAPI auto-documents validation errors.

If you want to override or formalize:

```
422: {
  "description": "Validation error",
  "content": {
    "application/json": {
      "example": {
        "detail": [
          {
            "loc": ["body", "email"],
            "msg": "field required",
            "type": "value_error.missing"
          }
        ]
      }
    }
  }
}
```

But usually you leave this alone unless you're building a very strict schema.

4 If You Want To Be Really Clean (Advanced)

Create a shared error mapping:

```
COMMON_ERRORS = {
    401: {"model": ErrorResponse, "description": "Unauthorized"},
    403: {"model": ErrorResponse, "description": "Forbidden"},
    500: {"model": ErrorResponse, "description": "Internal server error"},
}
```

Then reuse:

```
@router.get(
    "/users/{user_id}",
    response_model=UserResponse,
    responses={**COMMON_ERRORS, 404: {"model": ErrorResponse}},
)
```

Now your API is:

- DRY
- Scalable
- Easy to maintain

5 Global Exception Handlers (Clean Architecture Move)

If you want strong architecture points:

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

app = FastAPI()

@app.exception_handler(ValueError)
async def value_error_handler(request: Request, exc: ValueError):
    return JSONResponse(
        status_code=400,
        content={"detail": str(exc)},
    )
```

This lets you:

- Raise `valueError()` in domain layer
- Convert it cleanly at API boundary
- Keep layers separate

That's clean API design.

What Examiners Like To See

If you want solid marks:

- ✓ Consistent error format
 - ✓ Documented error codes in Swagger
 - ✓ Proper status codes (not everything 200...)
 - ✓ Meaningful descriptions
 - ✓ Authentication errors documented
-

Quick Rule of Thumb

If your endpoint can:

- Fail auth → document 401
- Fail permissions → document 403
- Not find resource → document 404
- Reject bad input → 400
- Validation fail → 422
- Server explode → 500

If you don't document it, it doesn't count.

If you want, show me one of your routes and I'll tighten it up to look like a distinction-level submission.

Grouping Routes FastAPI

03/13/2026 21:07:10

How can i group routes in the doc like having an auth section etc

If you're using FastAPI, grouping routes properly is exactly what APIRouter is for. If...

How can you make the docs look better like have a side bar to click through the...

Good. Now you're thinking beyond "it works" and into "this looks professional"....

Can i disable swagger and just use redoc at /docs

Yes. And you probably should if you want it to look cleaner. FastAPI lets you disable...



How can i group routes in the doc like having an auth section etc



If you're using **FastAPI**, grouping routes properly is exactly what `APIRouter` is for. If you don't group them, your docs turn into a messy wall of endpoints. Not a good look in an oral exam.

Given the brief requires clear API documentation and structured endpoints

 COMP3011_Coursework1_Brief__202... , you absolutely want clean sections like:

- Auth
- Users
- Listings
- Analytics
- Admin

Here's how to do it properly.

✓ The Clean Way (FastAPI)

1 Create separate routers

app/routes/auth.py

```
from fastapi import APIRouter

router = APIRouter(
    prefix="/auth",
    tags=["Authentication"]
)

@router.post("/login")
def login():
    return {"message": "Login"}

@router.post("/register")
```

```
def register():  
    return {"message": "Register"}
```

app/routes/users.py

```
from fastapi import APIRouter  
  
router = APIRouter(  
    prefix="/users",  
    tags=["Users"]  
)  
  
@router.get("/")  
def list_users():  
    return []
```

2 Include them in main.py

```
from fastapi import FastAPI  
from app.routes import auth, users  
  
app = FastAPI(title="My API")  
  
app.include_router(auth.router)  
app.include_router(users.router)
```

What This Does

In Swagger docs:

- Everything with `tags=["Authentication"]` appears under an **Authentication** section
- Everything with `tags=["Users"]` appears under a **Users** section

Clean. Professional. Examiner-friendly.

🔥 If You Want It Even Cleaner

You can define metadata for better descriptions:

```
app = FastAPI(
    title="Housing Insights API",
    openapi_tags=[
        {"name": "Authentication", "description": "Login and registration endpoints"},
        {"name": "Users", "description": "User management operations"},
        {"name": "Listings", "description": "CRUD operations for listings"},
    ]
)
```

Now your docs look deliberate instead of accidental.

🧠 If You're Using Django REST Framework

Use `@swagger_auto_schema` with tags:

```
@swagger_auto_schema(tags=["Authentication"])
```

🏆 Pro Tip for Higher Marks

Don't just group by model.

Group by **domain responsibility**:

- `auth`
- `analytics`
- `admin`
- `public`
- `internal`

That shows architectural thinking — which pushes you into the 70–80+ band.

If you tell me your stack (FastAPI? Django? Express?), I'll tailor it properly.



How can you make the docs look better like have a side bar to click through the sections and group the returns types with their endpoints



Good. Now you're thinking beyond "it works" and into "this looks professional". That's how you move from 60s to 80s.

If you're using **FastAPI**, Swagger UI is fine for dev... but it's not impressive for coursework.

You want something that:

- Has a **sidebar**
- Groups endpoints clearly
- Shows request + response models cleanly
- Looks like a real product

Here's how to level it up properly.

Option 1 — Use ReDoc (Instant Upgrade)

FastAPI already ships with **ReDoc**.

It gives you:

- Clean left-hand sidebar
- Section grouping
- Better layout
- Proper model schemas grouped under each endpoint

Just visit:

/redoc

If you disabled it, re-enable:

```
app = FastAPI(  
    docs_url="/docs",
```

```
    redoc_url="/documentation"
)
```

ReDoc already gives you:

- Sidebar navigation
- Tag-based grouping
- Expandable response models
- Schema references at bottom

For coursework? This is already miles better than default Swagger.

Option 2 — Make Swagger Look Better

Swagger UI can also be customised.

```
app = FastAPI(
    swagger_ui_parameters={
        "docExpansion": "none", # Collapse endpoints by default
        "defaultModelsExpandDepth": 2,
        "displayRequestDuration": True
    }
)
```

That stops it looking like a cluttered dump.

Group Return Types Properly (Important)

Right now, if your responses aren't typed, your docs look amateur.

Wrong:

```
@app.get("/users")
def get_users():
    return [{"name": "Morgan"}]
```

Professional:

```

from pydantic import BaseModel
from typing import List

class UserResponse(BaseModel):
    id: int
    email: str

@app.get("/users", response_model=List[UserResponse])
def get_users():
    ...

```

Now in docs:

- Response schema is clearly shown
- Models are grouped under "Schemas"
- It looks intentional

Examiners notice this immediately.

🔥 Option 3 — Use Custom OpenAPI + External Docs (This Looks Elite)

You can export OpenAPI:

```
curl http://localhost:8000/openapi.json > openapi.json
```

Then render it with:

★ **Redocly**

★ **Stoplight Elements**

★ **Scalar (very modern)**

These give:

- Collapsible sidebar
- Clean typography
- Better response grouping
- Production-style layout

For coursework? This screams polish.

Option 4 — Group by Tags Properly

Tags control sidebar sections.

```
router = APIRouter(  
    prefix="/auth",  
    tags=["Authentication"]  
)
```

If you want nested logic, fake hierarchy:

```
tags=["Analytics - Housing"]  
tags=["Analytics - Trends"]  
tags=["Admin"]
```

ReDoc will show them grouped alphabetically.

Option 5 — Document Response Variants Cleanly

Instead of returning random 400 errors, document them:

```
@app.post(  
    "/auth/login",  
    response_model=TokenResponse,  
    responses={  
        401: {"model": ErrorResponse},  
        400: {"model": ErrorResponse}  
    }  
)
```

Now your docs show:

- 200 → TokenResponse
- 400 → ErrorResponse
- 401 → ErrorResponse

That's professional API design.

If You Want To Impress the Marker

Do this:

1. Use ReDoc
2. Use response_model everywhere
3. Use proper error models
4. Custom OpenAPI tags metadata
5. Host docs publicly (e.g., Render/Fly.io)

Then mention in your presentation:

"Documentation is generated from typed models, ensuring consistency between implementation and contract."

That's how you sound like you know what you're doing.

If you tell me your exact stack (FastAPI I assume?) I can give you a setup that looks genuinely production-grade.



Can i disable swagger and just use redoc at /docs



Yes. And you probably should if you want it to look cleaner.

FastAPI lets you disable Swagger entirely and mount ReDoc wherever you want.

Here's the clean setup.

✅ Disable Swagger and Serve ReDoc at /docs

```
from fastapi import FastAPI

app = FastAPI(
    docs_url=None,          # Disable Swagger UI
    redoc_url="/docs",      # Serve ReDoc at /docs
    openapi_url="/openapi.json"
)
```

That's it.

Now:

- /docs → ReDoc (clean sidebar UI)
- /openapi.json → OpenAPI schema
- /docs (Swagger) → gone



Why This Is Better

ReDoc:

- Proper left-hand sidebar
- Cleaner model grouping
- Better readability
- Looks more "enterprise"

Swagger:

- Good for testing
- Cluttered for presentation

For coursework demo? ReDoc looks more polished.

Want It Even Cleaner?

You can also:

```
app = FastAPI(  
    title="Housing Insights API",  
    description="Advanced housing analytics and CRUD platform",  
    version="1.0.0",  
    docs_url=None,  
    redoc_url="/docs"  
)
```

Makes it feel intentional, not default.

One Thing to Remember

ReDoc is read-only.

If you rely on Swagger's "Try it out" feature during demo, you'll lose that.

Solution:

- Keep Swagger locally during dev
- Disable it for final deployment

If you want, I can also show you how to self-host a fully customised ReDoc page with branding, custom CSS, etc. That's how you push into 80+ territory.

Live Linking Table Setup

03/13/2026 21:02:00



Can you create “live” connecting tables for relationships. I have sale transactions that have an address and epc which have address and uprn. I want to have a table that actively updates when either tables have new records added



Right — you want a **live linking table** between:

- **Sale Transactions** (address only)
- **EPC records** (address + UPRN)
- And ideally end up with a relationship table like:

```
sale_id ↔ epc_id ↔ uprn
```

that updates automatically when *either* source table changes.

Good. This is exactly the kind of thing that can push your coursework into the 70–85+ band if you do it cleanly (see the advanced architecture + testing expectations in the brief

 COMP3011_Coursework1_Brief__202...).

Let’s break it down properly.

⚠ First: Don’t Use “Manual Sync Scripts”

If you’re thinking:

“I’ll run a background job every night that re-links stuff”

That works... but it’s not elegant.

You want **reactive linking**, not batch reprocessing.

✅ Best Solutions (Ranked)

🚩 Option 1 — Database-Level Triggers (Most Robust)

Use PostgreSQL triggers so that whenever:

- A new sale is inserted
- A new EPC is inserted

Postgres automatically attempts to match them and insert into a `property_links` table.

This keeps it:

- Fast
- ACID safe
- Centralised in the DB
- Stack-agnostic (FastAPI doesn't care)

🧱 Suggested Schema

`sales`

```
CREATE TABLE sales (  
    id SERIAL PRIMARY KEY,  
    address TEXT NOT NULL,  
    postcode TEXT,  
    sale_price INT,  
    sale_date DATE  
);
```

`epc`

```
CREATE TABLE epc (  
    id SERIAL PRIMARY KEY,  
    address TEXT NOT NULL,  
    postcode TEXT,  
    uprn BIGINT,  
    rating TEXT  
);
```

property_links

```
CREATE TABLE property_links (  
    sale_id INT REFERENCES sales(id) ON DELETE CASCADE,  
    epc_id INT REFERENCES epc(id) ON DELETE CASCADE,  
    uprn BIGINT,  
    PRIMARY KEY (sale_id, epc_id)  
);
```

Matching Strategy

You have 3 options:

1. Exact Address Match (Weak)

LOWER(address)

2. Postcode + Normalised Address (Better)

3. UPRN-Based Matching (Best)

If EPC has UPRN and you use **Open UPRN dataset**, then:

- First resolve sale → UPRN
- Then join by UPRN

That's the grown-up way.

Trigger-Based Auto Linking

Step 1 — Function

```
CREATE OR REPLACE FUNCTION link_sale_to_epc()  
RETURNS TRIGGER AS $$  
BEGIN  
    INSERT INTO property_links (sale_id, epc_id, uprn)  
    SELECT NEW.id, e.id, e.uprn  
    FROM epc e
```

```
WHERE LOWER(e.address) = LOWER(NEW.address)
ON CONFLICT DO NOTHING;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Step 2 — Trigger on Sales Insert

```
CREATE TRIGGER after_sale_insert
AFTER INSERT ON sales
FOR EACH ROW
EXECUTE FUNCTION link_sale_to_epc();
```

Step 3 — Mirror Trigger for EPC Insert

```
CREATE OR REPLACE FUNCTION link_epc_to_sale()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO property_links (sale_id, epc_id, uprn)
    SELECT s.id, NEW.id, NEW.uprn
    FROM sales s
    WHERE LOWER(s.address) = LOWER(NEW.address)
    ON CONFLICT DO NOTHING;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER after_epc_insert
AFTER INSERT ON epc
FOR EACH ROW
EXECUTE FUNCTION link_epc_to_sale();
```

Now It's "Live"

- Insert sale → links if EPC exists
- Insert EPC → links if sale exists
- No cron job
- No worker queue
- Fully transactional

Option 2 — Materialized View (Not Fully Live)

You could do:

```
CREATE MATERIALIZED VIEW property_links AS
SELECT ...
```

But it needs:

```
REFRESH MATERIALIZED VIEW property_links;
```

So it's not live unless you schedule refreshes.

Not ideal.

Option 3 — Application-Level Linking (Celery Worker)

Since you're already using:

- FastAPI
- Celery
- PostgreSQL
- Kubernetes

You *could*:

- Emit an event after insert
- Worker consumes event
- Worker links records

That's cool architecturally...

...but for coursework, DB triggers are cleaner and easier to defend in your viva.

The “Distinction-Level” Version

If you want to push this into 80+ territory:

Use:

- `pg_trgm` extension for fuzzy matching
- Indexed normalised address column
- UPRN-first matching logic
- Fallback to fuzzy address match

Example:

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE INDEX epc_address_trgm_idx
ON epc USING gin (address gin_trgm_ops);
```

Then match with:

```
WHERE similarity(e.address, NEW.address) > 0.85
```

Now you’re doing serious data engineering.

Architecturally Clean Pattern

Even better:

Instead of storing raw address as linking key:

Create a canonical `properties` table:

```
properties
-----
id
uprn
```

```
normalised_address  
postcode
```

Then:

```
sales → property_id  
epc   → property_id
```

This is the correct relational model.

Not sale ↔ epc.

Both should link to property.

That's what Land Registry + EPC integration systems do in production.

My Honest Recommendation

Given your stack and your tendency to over-engineer (in a good way):

Do this:

1. Create `properties` table
2. Use Open UPRN dataset as authoritative key
3. Insert sales → resolve to property
4. Insert EPC → resolve to property
5. Foreign key relationships
6. Add triggers only for resolving missing links

That becomes a clean star schema.

Why This Is Viva Gold

If asked:

Why did you design it this way?

You can say:

- Ensures referential integrity
- Avoids duplication

- Uses canonical identifier (UPRN)
- Keeps linking logic at DB level for atomicity
- Avoids eventual consistency issues

Examiners love that.

If you want, I can next:

- Design the full schema properly for your three UK datasets
- Or show you how to implement this cleanly in SQLAlchemy with async Postgres
- Or show how to test trigger logic properly

Your move.



Is there a way to do this en masse during initial population



Yes. And you absolutely should.

Triggers are great for incremental updates, but during **initial population of tens of millions of rows**, triggers are:

- Slow
- Transaction-heavy
- Index-churning
- Basically asking Postgres to suffer

For bulk linking, you want **set-based operations**, not row-by-row logic.



The Correct Way: Bulk Linking with INSERT...SELECT

Instead of letting triggers fire per row, do one big relational join.

Step 1 — Disable triggers (if already created)

```
ALTER TABLE sales DISABLE TRIGGER ALL;  
ALTER TABLE epc DISABLE TRIGGER ALL;
```

Step 2 — Bulk Link in One Statement

If matching by address:

```
INSERT INTO property_links (sale_id, epc_id, uprn)  
SELECT s.id, e.id, e.uprn  
FROM sales s  
JOIN epc e  
  ON LOWER(s.address) = LOWER(e.address)  
ON CONFLICT DO NOTHING;
```

That runs in one execution plan. No per-row overhead.

If matching by UPRN (much better):

```
INSERT INTO property_links (sale_id, epc_id, uprn)
SELECT s.id, e.id, e.uprn
FROM sales s
JOIN epc e
  ON s.uprn = e.uprn
WHERE s.uprn IS NOT NULL
ON CONFLICT DO NOTHING;
```

This is fast if indexed.



Performance Optimisation for Massive Datasets

Given you're working with:

- Price Paid Data (tens of millions)
- EPC (millions)
- OpenUPRN (tens of millions)

You need to do this properly.



1. Index Before Joining

```
CREATE INDEX idx_sales_address ON sales (LOWER(address));
CREATE INDEX idx_epc_address ON epc (LOWER(address));
```

Better:

```
CREATE INDEX idx_sales_uprn ON sales (uprn);
CREATE INDEX idx_epc_uprn ON epc (uprn);
```

Without indexes, you'll get a sequential join and your laptop will start sweating.



⚡ 2. Use UNLOGGED Table During Linking (Optional Turbo Mode)

If `property_links` is being built fresh:

```
CREATE UNLOGGED TABLE property_links (...);
```

After linking:

```
ALTER TABLE property_links SET LOGGED;
```

Speeds up initial population significantly.

⚡ 3. Drop Indexes Before Massive Inserts

If you're inserting 30M rows:

- Drop secondary indexes
- Insert
- Recreate indexes

Postgres builds indexes faster in bulk than maintaining them per insert.

🧠 Best Architecture for Initial Load

Here's how production systems do it:

Phase 1 — Load Raw Data

- COPY into staging tables
- No triggers
- No FKs

Phase 2 — Clean & Normalise

- Standardise addresses
- Resolve UPRNs
- Remove duplicates

Phase 3 — Bulk Link

- INSERT...SELECT join
- Use ON CONFLICT DO NOTHING

Phase 4 — Add Constraints & Triggers

- Enable FK constraints
- Enable incremental linking triggers

This gives you:

- Fast initial ingest
- Clean incremental updates afterwards

Even Cleaner: Don't Link Sale ↔ EPC

Since you're building House Stats 2.0 properly...

The relationally correct design is:

```
properties
-----
id
uprn
normalised_address
postcode

sales
-----
property_id FK

epc
-----
property_id FK
```

Then bulk linking becomes:

```
UPDATE sales s
SET property_id = p.id
```

```
FROM properties p
WHERE s.uprn = p.uprn;
```

Same for EPC.

No linking table needed.

Cleaner.

Faster.

More defensible in viva.

For Your Coursework Marking

If you explain:

- You used set-based relational operations instead of row triggers
- You disabled constraints during ingest
- You used indexed joins for bulk reconciliation

That hits:

- Code Quality & Architecture
- Performance awareness
- Advanced database design
- Professional deployment thinking

Which is comfortably 70–85 territory.

One More Pro Tip

If datasets are massive and you hit memory pressure:

Do batched linking:

```
INSERT INTO property_links (...)
SELECT ...
FROM sales s
JOIN epc e
  ON ...
WHERE s.id BETWEEN 1 AND 1000000;
```

Loop ranges in application.

If you tell me:

- Approx row counts
- Whether you're matching by UPRN or address
- Whether you're on local Postgres or Azure

I'll give you a performance-optimised plan specifically for your stack.

Bulk Metering Alternatives

03/13/2026 21:04:21



What functionality will the stripe adapter need to do for metered billing, add new use, etc



Good — this is exactly the right question to be asking before you lock the abstraction in.

If you're doing **metered billing with Stripe**, your adapter is not "charge card £X". It's essentially a **usage accounting bridge** between your domain and Stripe's subscription + price + usage system.

Let's break this down properly.



First: What Stripe Model You're Actually Using

For metered billing, Stripe uses:

- **Customer**
- **Product**
- **Price** (with `recurring.usage_type = metered`)
- **Subscription**
- **Usage Records** (reported per billing period)
- Invoice automatically generated

So your adapter needs to translate:

"User used 134 API credits today"

into:

"Increment usage on subscription item XYZ by 134 units"



Your Stripe Adapter Responsibilities

Your adapter should be responsible for:



1 Customer Lifecycle

Required methods

```
create_customer(user_id, email) -> stripe_customer_id  
get_customer(stripe_customer_id)  
delete_customer(stripe_customer_id)
```

You do NOT want Stripe IDs leaking all over your domain. Store them in your DB.

2 Subscription Management (Metered Plan)

```
create_subscription(customer_id, price_id) -> subscription_id  
cancel_subscription(subscription_id)  
get_subscription(subscription_id)
```

For metered billing, the subscription will contain:

```
subscription.items.data[0].id # <-- THIS is critical
```

You need the **subscription_item_id** to report usage.

3 Usage Reporting (Core Metered Functionality)

This is the big one.

```
record_usage(subscription_item_id, quantity, timestamp=None)
```

Internally:

```
stripe.SubscriptionItem.create_usage_record(  
    subscription_item_id,  
    quantity=quantity,  
    timestamp=int(time.time()),
```



```
    action="increment"  
)
```

Important:

- `action="increment"` → adds to existing usage
- Stripe accumulates usage during billing period
- Invoice is generated automatically

4 Webhook Handling (Non-Optional)

If you don't implement webhooks, your billing state will drift and you'll regret it.

Your adapter must handle:

- `invoice.paid`
- `invoice.payment_failed`
- `customer.subscription.deleted`
- `customer.subscription.updated`

So you'll want:

```
handle_webhook(event_payload)
```

And then internally:

```
if event.type == "invoice.paid":  
    mark_account_active()  
elif event.type == "invoice.payment_failed":  
    mark_account_past_due()
```

This is how your system stays consistent with Stripe's state.

5 Usage Querying (Optional but Useful)

Stripe does not give you real-time usage easily.

You might want:

```
get_current_usage(subscription_item_id)
```

But realistically?

You should track usage in your own DB and only push aggregated deltas to Stripe.

Stripe is not your source of truth.

What Your Adapter Interface Should Actually Look Like

Don't design it around Stripe. Design it around your domain.

Example:

```
class PaymentBase(ABC):

    def start_subscription(self, user_id: UUID, plan: PlanType):
        ...

    def cancel_subscription(self, user_id: UUID):
        ...

    def record_usage(self, user_id: UUID, units: int):
        ...

    def handle_webhook(self, payload: dict):
        ...
```

Then your `StripeAdapter` implements it.

Notice:

- No Stripe IDs in the public interface
- All Stripe junk is internal
- Domain speaks in users + plans + units

That's clean architecture.

What "Metered Billing" Actually Means for You

You need to decide:

Billing Model	Stripe Setup
Per API call	unit = 1 per request
Per GB transferred	unit = bytes or MB
Per compute minute	unit = minute
Credit-based	unit = 1 credit

Stripe only cares about **quantity**.

You define the unit.

Critical Design Advice

Since you're building a serious API system:

Don't report usage per request

Bad:

API hit → **call** Stripe immediately

That will:

- Destroy performance
- Add network latency
- Increase Stripe API rate risk

Instead:

1. Track usage internally (Postgres)
2. Aggregate usage per X minutes
3. Background worker pushes usage to Stripe

Since you're already using Celery, this fits perfectly.

Architecture Example

API Request



Increment usage counter **in** DB



Celery task **every** 5 mins



Push aggregated usage **to** Stripe



Reset **local** counter

That's scalable.



Advanced Features (If You Want 80+ Coursework Marks)

If you want this to look polished:

- Enforce usage limits before Stripe invoice
- Soft-limit warnings at 80%
- Hard stop at limit
- Grace period after payment failure
- Stripe Customer Portal integration
- Billing analytics endpoint

Now it's a product, not a homework API.



Testing Strategy (Important)

Stripe is a black box.

You should:

- Use Stripe test mode
- Mock Stripe client in unit tests
- Integration test against Stripe sandbox

Your adapter makes this easy.

Minimum Functionality Your Stripe Adapter Must Support

At absolute minimum for metered billing:

- Create customer
- Create subscription (metered price)
- Record usage
- Cancel subscription
- Handle invoice webhooks

If any of those are missing, your billing is incomplete.

Final Reality Check

Metered billing is not “just add Stripe”.

It is:

- Usage accounting
- Async reporting
- State reconciliation
- Failure handling

If you design the adapter lazily, it will bite you later.

If you design it properly now, it becomes a plug-and-play billing engine.

If you want, tell me:

- What exactly you're billing for (API calls? compute? data?)
- Whether you're doing free tier
- Whether you want tiered pricing

And I'll help you design the cleanest possible implementation.



How can you do bulk metering they only allow uploads via s3 which i don't want to do



Short answer: **Stripe does *not* provide a native “bulk API call” you can fire once with a huge list of usage events other than the S3 CSV upload** they document. The only ways Stripe supports bulk-style ingestion are:

- **CSV upload via S3/Stripe Dashboard** — normally what folks mean by “bulk upload,” but you said you *don't want S3* docs.stripe.com
- **Individual *usage events* via API** — one call per event or per aggregated value; there's no multi-record API endpoint that accepts an array in one HTTP request docs.stripe.com [+1](#)

Stripe's docs clearly show the API approach — you create a usage record for a subscription item with a single quantity per API call — and mention batching *on your side*, not at Stripe's side docs.stripe.com .

So the real trick isn't a “bulk API” but how you **batch on your side before submitting**.

Why Stripe Doesn't Give You a Proper Bulk API

Stripe's design:

- API is rate-limited (~100 calls/sec) and doesn't let you post arrays of usage points in one request [Stack Overflow](#)
- S3/CSV upload exists *only* because some customers have huge datasets they want to push in once without hitting API limits docs.stripe.com
- There is no documented API endpoint to upload a JSON list of usage records in a single request

So you have two options if you *can't or don't want* an S3 workflow:

Practical Alternatives (Without S3)

✅ 1. Aggregate Before You Send

The consensus best practice is:

- Collect usage events in your own datastore

- **Aggregate them into larger chunks** (e.g., per user per hour/day)
- For each chunk, send **one** usage record to Stripe

This is effectively “batching” on your side — Stripe treats each usage record as a single API call, but the value you send can represent 10k events:

```
POST /v1/billing/meter_events
{
  event_name: "api_calls",
  payload_value: 10000,
  payload_stripe_customer_id: "cus_...",
  timestamp: 1712345678
}
```

Later Stripe combines these into totals for billing docs.stripe.com.

This is how everyone handles high volume without S3.

✓ 2. Internal Buffer + Background Worker

Architect it like:

```
API request → increment local counter
Every X minutes → push aggregated counts to Stripe
```

This gives:

- fewer API calls
- less risk of hitting rate limits
- you control batching logic

Patterns:

- Aggregate by customer + meter + hour/day
- Use a queue/worker to push usage
- Retry on failure

This mimics S3 bulk uploads in a way that Stripe supports.

🚫 3. There Is No Stripe Multi-Record API

People on StackOverflow confirm that Stripe has:

"no way to batch a list of usage records in one request" — you must either send many calls, or aggregate locally first [Stack Overflow](#) .

So you can't call one endpoint with 1000 events.

How This Would Look In Practice (Real-World)

Your system:

```
usage_records = {  
  user1: { 'meterA': 150, 'meterB': 300 },  
  user2: { 'meterA': 80, 'meterB': 20 }  
}  
  
for each (user, meter):  
  send one API call to Stripe with aggregated quantity
```

Even if you have millions of raw events, you *reduce the calls* by batching in your app.

What You *Can* Still Do

Stripe recent docs talk about a **Meter Usage Analytics API** (preview) to query aggregated usage, but that's for reading — not submitting ✨ docs.stripe.com .

There have also been 3rd-party integrations (e.g., OpenMeter) that help sync data out of infra into Stripe without you writing the ingestion pipeline — but that's a different product, not a Stripe API feature [OpenMeter](#) .

Summary — Reality Check

- 🔴 Stripe doesn't let you upload a JSON list of usage records in one API call.
- 🟡 S3/CSV is the only true "bulk upload," but you don't want that.
- ✅ Your only scalable pattern is *internal aggregation + fewer API calls*, e.g. 1 usage record per customer/meter per period instead of one per event.

That's the practical, supported option if you're avoiding S3.

If you want, tell me:

- How many usage events you're generating per minute
- Whether usage is per customer or per subscription item

I can sketch an efficient batching strategy and rate-limit-safe implementation for your backend.

SQL LIKE Indexing

03/13/2026 21:06:22



Can you create indexes for SQL LIKE queries?



Yes. But it depends **how** you're using `LIKE`.

Here's the straight answer:

1 Standard B-Tree Index (Works Sometimes)

If your query looks like this:

```
SELECT *  
FROM properties  
WHERE address LIKE 'LS2%';
```

You **can** use a normal index:

```
CREATE INDEX idx_address ON properties(address);
```

This works because the pattern has a **fixed prefix** (`'LS2'`).

The database can use the index to jump to that range.

✅ Works:

```
LIKE 'abc%'
```

❌ Does NOT use index:

```
LIKE '%abc'  
LIKE '%abc%'
```

Why? Because with a leading wildcard, the database has to scan everything.

2 PostgreSQL-Specific: `text_pattern_ops`

If you're using PostgreSQL (you are for House Stats), it's safer to do:

```
CREATE INDEX idx_address_pattern
ON properties (address text_pattern_ops);
```

This ensures prefix `LIKE 'abc%'` queries always use the index.

3 Best Solution for `%something%` → Trigram Index (Postgres)

For full substring search like:

```
WHERE address LIKE '%leeds%'
```

Use `pg_trgm`.

Enable extension:

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
```

Create trigram index:

```
CREATE INDEX idx_address_trgm
ON properties
USING gin (address gin_trgm_ops);
```

Now `%leeds%` queries are fast.

This is what you want if:

- You're doing search bars
- Fuzzy matching addresses

- EPC ↔ PPD linking via strings
- Autocomplete

For your housing dataset (millions of rows), trigram indexing is a **no-brainer**.

4 If You're Doing Case-Insensitive Search

Instead of:

```
WHERE LOWER(address) LIKE 'leeds%'
```

Do this:

```
CREATE INDEX idx_address_lower  
ON properties (LOWER(address) text_pattern_ops);
```

Or combine with trigram:

```
CREATE INDEX idx_address_trgm_lower  
ON properties  
USING gin (LOWER(address) gin_trgm_ops);
```

5 Performance Summary

Query Type	Index Type
LIKE 'abc%'	B-tree
LIKE '%abc%'	GIN + pg_trgm
Case insensitive prefix	Functional B-tree
Case insensitive substring	GIN trigram

What I'd Do in Your Housing Stats API

You're linking:

- PPD
- EPC
- UPRN

If you're doing **address matching**, use:

```
CREATE EXTENSION pg_trgm;

CREATE INDEX idx_epc_address_trgm
ON epc_certificates
USING gin (address gin_trgm_ops);

CREATE INDEX idx_ppd_address_trgm
ON price_paid
USING gin (address gin_trgm_ops);
```

Otherwise you'll be doing full table scans on tens of millions of rows. That's suicide performance-wise.

If you want, tell me:

- Row count
- Example query
- Postgres version

And I'll tell you exactly what index strategy you should use.